

# Set A — Practice for Problem 1

---

## A1. Library Loan Tracker

### Problem Breakdown / Example

```
# Book
# - title (str)
# - isbn (int)
# - is_checked_out (bool)

# Member
# - borrow(book) -> function
# - return_book(book) -> function

# BookUnavailableError
# - Custom Exception
# - condition: cannot borrow an unavailable book
# - message must include the book title

# Task:
# 1. Add __str__ on Book -> shows title and availability
#
# Results:
# 1. successful borrow
# 2. successful return
# 3. failed borrow attempt on an already checked-out book
# -> catch the custom exception
# -> print a friendly message instead of a traceback
```

### Code

```
# Custom exception for when someone tries to borrow
# a book that is already checked out.
class BookUnavailableError(Exception):
    pass
```

```

# Book class stores information about one book.
class Book:
    def __init__(self, title, isbn):
        self.title = title          # Book title
        self.isbn = isbn            # Book ISBN
        self.is_checked_out = False # New books are available

    # __str__ controls what is displayed when we print the book.
    def __str__(self):
        if self.is_checked_out:
            availability = "Checked out"
        else:
            availability = "Available"

        return f"{self.title} - {availability}"

```

```

# Member class represents a library member.
class Member:
    def __init__(self, name):
        self.name = name

    # Borrow a book.
    def borrow(self, book):
        # Check if the book is already checked out.
        if book.is_checked_out:
            # Raise our custom exception if unavailable.
            raise BookUnavailableError(
                f"'{book.title}' is already checked out."
            )

        # Mark the book as checked out.
        book.is_checked_out = True

        print(f"{self.name} successfully borrowed '{book.title}'.")

    # Return a book.
    def return_book(self, book):
        # Mark the book as available again.
        book.is_checked_out = False

        print(f"{self.name} successfully returned '{book.title}'.")

```

```

# -----
# Demonstration
# -----

book1 = Book("Python Basics", 12345)
member1 = Member("Alice")

# Show the book before borrowing.
print(book1)

# 1. Successful borrow
member1.borrow(book1)

# Show the book after borrowing.
print(book1)

# 2. Successful return
member1.return_book(book1)

# Show the book after returning.
print(book1)

# Borrow it again so we can demonstrate the error.
member1.borrow(book1)

# 3. Failed borrow attempt
try:
    member1.borrow(book1)

# Catch only our custom exception.
except BookUnavailableError as e:
    print(f"Borrow failed: {e}")

```

## What the code demonstrates

1. A Book starts as Available.
2. borrow() changes is\_checked\_out to True.
3. return\_book() changes is\_checked\_out back to False.
4. Trying to borrow an already checked-out book raises BookUnavailableError.
5. try/except catches the error and prints a friendly message.
6. \_\_str\_\_ makes printing a Book easy to understand.

---

## A2. Shape Hierarchy with a Twist

### Problem Breakdown / Example

```
# Shape
# - Abstract Base Class
# - inherits from ABC
# - area() -> abstractmethod
# - perimeter() -> abstractmethod
#
# Circle
# - radius must be > 0
#
# Rectangle
# - length must be > 0
# - width must be > 0
#
# Triangle
# - side_a must be > 0
# - side_b must be > 0
# - side_c must be > 0
#
# Task:
# 1. Prevent Shape from being instantiated directly
# 2. Every subclass must implement area()
# 3. Every subclass must implement perimeter()
# 4. Reject non-positive dimensions with ValueError
# 5. Create total_area(shapes)
# 6. Use polymorphism to calculate mixed shape areas
```

### Code

```
from abc import ABC, abstractmethod
import math

# Shape is an abstract base class.
# ABC means this class is meant to be inherited from.
class Shape(ABC):
```

```
# Any subclass MUST implement area().
```

```
@abstractmethod
```

```
def area(self):
```

```
    pass
```

```
# Any subclass MUST implement perimeter().
```

```
@abstractmethod
```

```
def perimeter(self):
```

```
    pass
```

```
# Circle inherits from Shape.
```

```
class Circle(Shape):
```

```
    def __init__(self, radius):
```

```
        # Radius must be positive.
```

```
        if radius <= 0:
```

```
            raise ValueError("Invalid radius: radius must be positive.")
```

```
        self.radius = radius
```

```
# Formula for circle area.
```

```
def area(self):
```

```
    return math.pi * self.radius ** 2
```

```
# Formula for circle perimeter.
```

```
def perimeter(self):
```

```
    return 2 * math.pi * self.radius
```

```
# Rectangle inherits from Shape.
```

```
class Rectangle(Shape):
```

```
    def __init__(self, length, width):
```

```
        # Validate both dimensions.
```

```
        if length <= 0:
```

```
            raise ValueError("Invalid length: length must be positive.")
```

```
        if width <= 0:
```

```
            raise ValueError("Invalid width: width must be positive.")
```

```
        self.length = length
```

```
        self.width = width
```

```
# Formula for rectangle area.
```

```

def area(self):
    return self.length * self.width

# Formula for rectangle perimeter.
def perimeter(self):
    return 2 * (self.length + self.width)

# Triangle inherits from Shape.
class Triangle(Shape):
    def __init__(self, side_a, side_b, side_c):
        # Validate every side.
        if side_a <= 0:
            raise ValueError("Invalid side_a: side_a must be positive.")

        if side_b <= 0:
            raise ValueError("Invalid side_b: side_b must be positive.")

        if side_c <= 0:
            raise ValueError("Invalid side_c: side_c must be positive.")

        # Save the sides.
        self.side_a = side_a
        self.side_b = side_b
        self.side_c = side_c

        # Optional extra validation:
        # A real triangle must satisfy the triangle inequality.
        if (
            side_a + side_b <= side_c
            or side_a + side_c <= side_b
            or side_b + side_c <= side_a
        ):
            raise ValueError("Invalid triangle: sides cannot form a triangle.")

    # Heron's formula for triangle area.
    def area(self):
        s = (self.side_a + self.side_b + self.side_c) / 2

        return math.sqrt(
            s
            * (s - self.side_a)
            * (s - self.side_b)
            * (s - self.side_c)

```

)

# Sum of all three sides.

def perimeter(self):

return self.side\_a + self.side\_b + self.side\_c

# This function demonstrates polymorphism.

# It does not need to know what specific shape each object is.

def total\_area(shapes: list[Shape]) -> float:

total = 0

# Every object is expected to have an area() method.

for shape in shapes:

total += shape.area()

return total

# -----

# Demonstration

# -----

circle = Circle(5)

rectangle = Rectangle(4, 6)

triangle = Triangle(3, 4, 5)

# Mixed list of different shape types.

shapes = [circle, rectangle, triangle]

print("Circle area:", circle.area())

print("Circle perimeter:", circle.perimeter())

print("Rectangle area:", rectangle.area())

print("Rectangle perimeter:", rectangle.perimeter())

print("Triangle area:", triangle.area())

print("Triangle perimeter:", triangle.perimeter())

# total\_area() works because every shape has area().

print("Total area:", total\_area(shapes))

# This would fail because Shape is abstract.

```
# Uncomment to test:
#
# shape = Shape()

# This would also fail because dimensions must be positive.
# Uncomment to test:
#
# bad_circle = Circle(-5)
```

## Important idea

Shape is abstract, so Shape() cannot be created directly.

Circle, Rectangle, and Triangle all provide their own versions of area() and perimeter().

The total\_area() function can work with all of them without checking:

```
if circle...
if rectangle...
if triangle...
```

That is polymorphism.

---

## A3. Retry-Safe API Caller

### Problem Breakdown / Example

```
# unreliable_call()
# - already given by the instructor
# - may randomly raise ConnectionError
#
# AllRetriesFailedError
# - Custom Exception
# - should include number of attempts made
#
# safe_call(func, max_retries=3)
```



```
# - wrapper function, NOT a decorator
# - call func()
# - catch ConnectionError specifically
# - log every failed attempt
# - try again
# - if all 3 attempts fail -> raise AllRetriesFailedError
```

## Code

```
import random
import logging
```

```
# Configure logging so failed attempts are displayed.
logging.basicConfig(level=logging.WARNING, format="%(message)s")
```

```
# Custom exception for when every attempt fails.
class AllRetriesFailedError(Exception):
    pass
```

```
# This is only a sample version of unreliable_call().
# In the exam, the instructor may already provide this function.
def unreliable_call():
    # Simulate a 50% chance of failure.
    if random.random() < 0.5:
        raise ConnectionError("Server connection failed.")

    return "API call successful!"
```

```
# Safe wrapper around a function.
def safe_call(func, max_retries=3):

    # Make sure the number of attempts is valid.
    if max_retries <= 0:
        raise ValueError("max_retries must be greater than 0.")

    # Try the function up to max_retries times.
    for attempt in range(1, max_retries + 1):

        try:
```

```

    # Attempt to call the function.
    result = func()

    # If it succeeds, immediately return the result.
    return result

except ConnectionError as error:
    # Log the failed attempt.
    logging.warning(
        f"Attempt {attempt}/{max_retries} failed: {error}"
    )

    # If this was the final attempt, raise our custom exception.
    if attempt == max_retries:
        raise AllRetriesFailedError(
            f"All {attempt} attempts failed."
        ) from error

# -----
# Demonstration
# -----

try:
    result = safe_call(unreliable_call, max_retries=3)
    print(result)

except AllRetriesFailedError as error:
    print(f"Final failure: {error}")

```

## What happens

Example if the first call succeeds:

API call successful!

Example if the first two fail but the third succeeds:

Attempt 1/3 failed: Server connection failed.  
 Attempt 2/3 failed: Server connection failed.  
 API call successful!

Example if all three fail:

Attempt 1/3 failed: Server connection failed.  
Attempt 2/3 failed: Server connection failed.  
Attempt 3/3 failed: Server connection failed.  
Final failure: All 3 attempts failed.

## Important exam point

Catch only `ConnectionError`.

Do NOT write:

`except Exception:`

because the problem specifically says to catch `ConnectionError`.

---

# A4. Employee Payroll with Class Attributes

## Problem Breakdown / Example

```
# Employee
# - tax_rate -> CLASS ATTRIBUTE
# - name -> INSTANCE ATTRIBUTE
# - base_salary -> INSTANCE ATTRIBUTE
#
# net_pay()
# - base_salary - tax
#
# from_dict(cls, data)
# - classmethod
# - creates Employee from dictionary
#
# Task:
# 1. Demonstrate class vs instance attributes
# 2. Change Employee.tax_rate once
# 3. Show all existing employees use the new rate
```

# Code

```
class Employee:

    # Class attribute.
    # Every Employee shares this value.
    tax_rate = 0.10

    def __init__(self, name, base_salary):
        # Instance attributes.
        # Each employee has their own values.
        self.name = name
        self.base_salary = base_salary

    # Calculate take-home pay.
    def net_pay(self):
        # Use Employee.tax_rate so the shared class value is used.
        tax = self.base_salary * Employee.tax_rate

        return self.base_salary - tax

    # Alternative constructor from a dictionary.
    @classmethod
    def from_dict(cls, data: dict):
        # cls refers to Employee.
        return cls(
            data["name"],
            data["base_salary"]
        )

# -----
# Demonstration
# -----

# Create employees normally.
employee1 = Employee("Alice", 30000)
employee2 = Employee("Bob", 40000)

print(employee1.name, employee1.net_pay())
print(employee2.name, employee2.net_pay())

# Create another employee from a dictionary.
```

```
employee_data = {  
    "name": "Charlie",  
    "base_salary": 50000  
}  
  
employee3 = Employee.from_dict(employee_data)  
  
print(employee3.name, employee3.net_pay())  
  
# Change the CLASS attribute once.  
Employee.tax_rate = 0.20  
  
print("\nAfter changing Employee.tax_rate to 20%:")  
  
# All existing employees now use the new class-level tax rate.  
print(employee1.name, employee1.net_pay())  
print(employee2.name, employee2.net_pay())  
print(employee3.name, employee3.net_pay())
```

## Example Result

Original tax rate = 10%

Alice -> 27000  
Bob -> 36000  
Charlie -> 45000

After changing tax rate to 20%

Alice -> 24000  
Bob -> 32000  
Charlie -> 40000

## Important exam point

tax\_rate is a CLASS attribute:

Employee.tax\_rate = 0.20

name and base\_salary are INSTANCE attributes:

```
self.name
self.base_salary
```

Also, strictly speaking, **changing the tax rate is not a raise**. It changes employees' net pay by changing the tax amount. The important concept the question is testing is that the shared class attribute changes for all existing objects.

---

## A5. Logging Decorator for a Grading Function

### Problem Breakdown / Example

```
# calculate_grade(score)
# - already works
# - original code must NOT be modified
#
# @log_grades
# - decorator
# - logs timestamp
# - logs function name
# - logs input
# - logs result
#
# Use:
# from functools import wraps
#
# Why wraps?
# - keeps the original function metadata
# - especially __name__
```

### Code

```
from functools import wraps
from datetime import datetime
```

```
# Decorator function.
def log_grades(func):
```

```

# wraps(func) preserves the original function's metadata,
# including its __name__.
@wraps(func)
def wrapper(*args, **kwargs):

    # Call the original function.
    result = func(*args, **kwargs)

    # Get the current timestamp.
    timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")

    # Print the requested log message.
    print(
        f'[{timestamp}] called {func.__name__} '
        f'with {args} -> {result}'
    )

    # Return exactly what the original function returned.
    return result

return wrapper

```

```

# Original grading function.
# We do not modify its logic.
@log_grades
def calculate_grade(score: float) -> str:

```

```

    if score >= 90:
        return "A"
    elif score >= 80:
        return "B"
    elif score >= 70:
        return "C"
    elif score >= 60:
        return "D"
    else:
        return "F"

```

```

# -----
# Demonstration
# -----

```

```
grade = calculate_grade(87)

print("Grade:", grade)

# Because we used @wraps(func),
# the function still reports its original name.
print("Function name:", calculate_grade.__name__)
```

## Example Result

```
[2026-09-15 01:00:00] called calculate_grade with (87,) -> B
Grade: B
Function name: calculate_grade
```

## Important exam comment

```
# Without functools.wraps, calculate_grade.__name__
# would normally become "wrapper" because the decorator
# replaced the original function with the wrapper function.
```

---

# Set B — Practice for Problem 2

---

## B1. Messy Survey Cleanup

### Problem Breakdown / Example

```
# Dataset
# - 8–10 rows
# - inconsistent whitespace
# - mixed-case Yes / yes / YES
# - missing age values
# - one duplicate row
#
# Task:
```



```
# 1. Put data into a DataFrame
# 2. Print describe() BEFORE cleaning
# 3. Strip whitespace
# 4. Normalize text
# 5. Fill missing age with mean
# 6. Remove exact duplicate rows
# 7. Print describe() AFTER cleaning
```

## Code

```
import pandas as pd
import numpy as np

# -----
# 1. Create a messy sample dataset
# -----

data = [
    {"name": " Alice ", "answer": " YES ", "age": 20},
    {"name": "Bob", "answer": "yes", "age": 22},
    {"name": " Charlie", "answer": "NO", "age": 21},
    {"name": "Diana ", "answer": " Yes", "age": np.nan},
    {"name": "Eve", "answer": " no ", "age": 25},
    {"name": "Frank", "answer": "YES", "age": 24},
    {"name": "Grace", "answer": " yes ", "age": np.nan},
    {"name": "Henry", "answer": "No", "age": 23},

    # Duplicate row
    {"name": " Alice ", "answer": " YES ", "age": 20},
]

# Convert the list of dictionaries into a DataFrame.
df = pd.DataFrame(data)

# -----
# 2. Describe BEFORE cleaning
# -----

print("==== BEFORE CLEANING ===")
print(df.describe(include="all"))
```

```

# -----
# 3. Clean text columns
# -----

# Strip whitespace from the name column.
df["name"] = df["name"].str.strip()

# Strip whitespace and convert answers to lowercase.
df["answer"] = df["answer"].str.strip().str.lower()


# -----
# 4. Fill missing age values
# -----

# Calculate the mean age.
mean_age = df["age"].mean()

# Fill NaN values with the mean.
df["age"] = df["age"].fillna(mean_age)


# -----
# 5. Remove exact duplicate rows
# -----

# drop_duplicates() removes rows that are completely identical.
df = df.drop_duplicates()


# -----
# 6. Describe AFTER cleaning
# -----

print("\n===== AFTER CLEANING =====")
print(df.describe(include="all"))


# Show the final cleaned data.
print("\n===== CLEANED DATA =====")
print(df)

```

## Main cleaning commands to remember

```
# Remove spaces
df["name"] = df["name"].str.strip()

# Convert text to lowercase
df["answer"] = df["answer"].str.lower()

# Do both
df["answer"] = df["answer"].str.strip().str.lower()

# Fill missing values
df["age"] = df["age"].fillna(df["age"].mean())

# Remove duplicate rows
df = df.drop_duplicates()
```

---

## B2. Vectorized Grading Curve

### Problem Breakdown / Example

```
# scores:
# [62, 78, 95, 88, 40, 99, 71]
#
# Task:
# 1. Add 5 to every score
# 2. Maximum result = 100
# 3. No for loop
# 4. No list comprehension
# 5. Use NumPy vectorization
#
# Possible tools:
# np.where()
# np.clip()
```

### Code

```
import numpy as np
```

```
# Original exam scores.
scores = np.array([62, 78, 95, 88, 40, 99, 71])

# Add 5 to every score.
curved_scores = scores + 5

# Cap every value at 100.
# Any value greater than 100 becomes 100.
curved_scores = np.clip(curved_scores, 0, 100)

print("Original scores:", scores)
print("Curved scores: ", curved_scores)

# Why is this faster?
#
# NumPy performs the operation using optimized array operations
# internally rather than repeatedly executing Python-level code
# for every element. This usually makes vectorized operations
# much faster for large arrays than Python loops.
```

## Result

```
Original scores: [62 78 95 88 40 99 71]
Curved scores:  [67 83 100 93 45 100 76]
```

## Alternative using `np.where()`

```
curved_scores = np.where(scores + 5 > 100, 100, scores + 5)

print(curved_scores)
```

---

## B3. Nearest Neighbor by Hand

## Problem Breakdown / Example

```
# Each customer has 3 features:
# [recency, frequency, monetary]
#
# Task:
# 1. Create 4 existing customer vectors
# 2. Create 1 new customer vector
# 3. Use sklearn cosine_similarity
# 4. Calculate similarity with all 4 customers
# 5. Sort highest to lowest
# 6. Print the closest customer
```

## Code

```
import numpy as np
from sklearn.metrics.pairwise import cosine_similarity

# -----
# Existing customer profiles
# -----

customers = np.array([
    [10, 5, 200], # Customer A
    [8, 6, 220],  # Customer B
    [30, 2, 100], # Customer C
    [12, 4, 180]  # Customer D
])

# New customer's profile.
new_customer = np.array([
    [9, 5, 210]
])

# Calculate cosine similarity between
# the new customer and all existing customers.
similarities = cosine_similarity(new_customer, customers)[0]

# Create customer labels.
```

```
customer_names = ["Customer A", "Customer B", "Customer C", "Customer D"]
```

```
# Combine names and scores.
```

```
results = list(zip(customer_names, similarities))
```

```
# Sort from highest similarity to lowest similarity.
```

```
results.sort(key=lambda x: x[1], reverse=True)
```

```
# -----
```

```
# Print all results
```

```
# -----
```

```
print("Similarity scores:")
```

```
for name, score in results:
```

```
    print(f'{name}: {score:.4f}')
```

```
# The first item after sorting is the closest customer.
```

```
closest_customer = results[0]
```

```
print(
```

```
    f'\nClosest match: {closest_customer[0]} "
```

```
    f'with similarity {closest_customer[1]:.4f}'
```

```
)
```

## Example interpretation

A cosine similarity closer to 1 means the vectors point in a more similar direction.

The customer with the HIGHEST similarity score is considered the closest match.

## Important exam pattern

```
cosine_similarity(new_customer, customers)
```

returns one similarity score for each existing customer.

Then:

```
results.sort(key=lambda x: x[1], reverse=True)
```

puts the highest score first.

---

## B4. Spam Filter Confusion Matrix

### Problem Breakdown / Example

```
# expected_labels
# - correct / real answer
#
# predicted_labels
# - spam filter prediction
#
# Task:
# 1. Create 10 examples
# 2. Include at least 2 mistakes
# 3. Compute confusion matrix
# 4. Compute precision
# 5. Compute recall
# 6. Compute F1 score
# 7. Use "spam" as the positive class
# 8. Explain false positive and false negative
```

### Code

```
from sklearn.metrics import (
    confusion_matrix,
    precision_score,
    recall_score,
    f1_score
)
```

```
# The correct labels for 10 emails.
```

```
expected_labels = [  
    "spam",  
    "ham",  
    "ham",  
    "spam",  
    "spam",  
    "ham",  
    "spam",  
    "ham",  
    "spam",  
    "ham"  
]
```

```
# The spam filter's predictions.  
# We intentionally include mistakes.  
predicted_labels = [  
    "spam",  
    "ham",  
    "spam", # Mistake: ham predicted as spam  
    "ham",  # Mistake: spam predicted as ham  
    "spam",  
    "ham",  
    "spam",  
    "spam", # Mistake: ham predicted as spam  
    "ham",  # Mistake: spam predicted as ham  
    "ham"  
]
```

```
# Calculate the confusion matrix.  
# Labels are ordered as:  
# [[ham, ham], [spam, spam]]  
cm = confusion_matrix(  
    expected_labels,  
    predicted_labels,  
    labels=["ham", "spam"]  
)
```

```
print("Confusion Matrix:")  
print(cm)
```

```
# Precision for the "spam" class.
```



```
precision = precision_score(  
    expected_labels,  
    predicted_labels,  
    pos_label="spam"  
)  
  
# Recall for the "spam" class.  
recall = recall_score(  
    expected_labels,  
    predicted_labels,  
    pos_label="spam"  
)  
  
# F1 score for the "spam" class.  
f1 = f1_score(  
    expected_labels,  
    predicted_labels,  
    pos_label="spam"  
)  
  
print("\nPrecision:", precision)  
print("Recall:", recall)  
print("F1 Score:", f1)
```

## Example Result

For the sample above:

Confusion Matrix:

```
[[3 2]  
 [2 3]]
```

Precision: 0.6

Recall: 0.6

F1 Score: 0.6

## Problem Explanation

A false positive happens when the email is actually legitimate but the spam filter incorrectly labels it as spam. For example, a real message from your professor could be moved to the spam folder.

A false negative happens when the email is actually spam but the filter incorrectly allows it to pass as a normal email. This could be especially dangerous if the message is phishing or malicious.

For a security-focused real-world product, I would prioritize minimizing false negatives because dangerous spam should not be allowed into the user's inbox.

## Confusion Matrix Meaning

[[TN, FP],  
[FN, TP]]

For this problem:

TN = legitimate email correctly identified as legitimate

FP = legitimate email incorrectly marked as spam

FN = spam email incorrectly marked as legitimate

TP = spam email correctly identified as spam

---

## B5. Token Budget Estimator

### Problem Breakdown / Example

# Given:

# max\_context = 8000

# reserved\_for\_output = 1000

#

# Available input budget:

# 8000 - 1000 = 7000 tokens

#

# Estimate each chunk with:

# len(text) // 4

#

# Task:

# 1. Count every chunk

# 2. Add all estimates

# 3. Compare total against available budget

- # 4. Raise ValueError if over budget
- # 5. Clearly say how far over budget
- # 6. Test once under budget
- # 7. Test once over budget

## Code

```
# Total model context window.
max_context = 8000

# Tokens we reserve for the model's response.
reserved_for_output = 1000

# Tokens available for input chunks.
available_budget = max_context - reserved_for_output

# Function that checks whether the chunks fit.
def estimate_token_budget(
    chunks,
    max_context=8000,
    reserved_for_output=1000
):
    # Calculate how many tokens are available for input.
    available_budget = max_context - reserved_for_output

    # Store individual chunk estimates.
    token_counts = []

    # Calculate token estimate for each chunk.
    for chunk in chunks:
        # Rough estimate from the reading guide.
        estimate = len(chunk) // 4

        # Save the estimate.
        token_counts.append(estimate)

    # Add all chunk estimates together.
    total_tokens = sum(token_counts)

    print("Individual chunk estimates:", token_counts)
    print("Total estimated tokens:", total_tokens)
    print("Available input budget:", available_budget)
```

```

# Check whether the total is over the budget.
if total_tokens > available_budget:

    # Calculate how far over budget we are.
    over_by = total_tokens - available_budget

    raise ValueError(
        f"Context budget exceeded by {over_by} tokens."
    )

# If we reach this point, the chunks fit.
print("Chunks fit within the available context budget.")

return total_tokens

```

```

# =====
# TEST 1: UNDER BUDGET
# =====

```

```

chunks_under = [
    "Python is a programming language used for data analysis.",
    "Pandas provides DataFrame structures for organizing data.",
    "NumPy provides fast numerical operations on arrays.",
    "Scikit-learn is commonly used for machine learning.",
    "Vectorization can make numerical operations faster.",
    "Cleaning data is an important preprocessing step."
]

```

```

print("==== UNDER BUDGET TEST ====")

```

```

try:
    estimate_token_budget(chunks_under)

```

```

except ValueError as error:
    print("Error:", error)

```

```

# =====
# TEST 2: DELIBERATELY OVER BUDGET
# =====

```

```

# Create six long sample chunks.
# Repeating text makes the total obviously exceed the budget.
chunks_over = [
    "This is a long document chunk. " * 5000,
    "This is another long document chunk. " * 5000,
    "Retrieved information from a knowledge base. " * 5000,
    "This chunk contains additional reference material. " * 5000,
    "More retrieved information for the language model. " * 5000,
    "Final large document chunk for testing. " * 5000
]

print("\n===== OVER BUDGET TEST =====")

try:
    estimate_token_budget(chunks_over)

except ValueError as error:
    # Catch the ValueError so the program does not crash
    # with a raw traceback.
    print("Error:", error)

```

## Important calculation

Maximum context:  
8000 tokens

Reserved for output:  
1000 tokens

Available input:  
 $8000 - 1000 = 7000$  tokens

The function therefore checks:

```
if total_tokens > 7000:
```

Conceptually, the core calculation is:

```
estimated_tokens = len(text) // 4
```

And the over-budget calculation is:

```
over_by = total_tokens - available_budget
```

---

## Quick Exam Pattern Summary

### A1 — Custom Exception + Classes

```
class MyError(Exception):  
    pass
```

```
if condition:  
    raise MyError("message")
```

```
try:  
    ...  
except MyError as e:  
    print(e)
```

### A2 — Abstract Base Class

```
from abc import ABC, abstractmethod
```

```
class Shape(ABC):  
  
    @abstractmethod  
    def area(self):  
        pass
```

### A3 — Retry

```
for attempt in range(1, max_retries + 1):  
    try:  
        return func()  
    except ConnectionError:  
        ...
```

## A4 — Class vs Instance Attribute

```
class Employee:
    tax_rate = 0.10    # class attribute

    def __init__(self, name):
        self.name = name # instance attribute
```

Class attribute is shared:

```
Employee.tax_rate = 0.20
```

## A5 — Decorator

```
from functools import wraps
```

```
def decorator(func):

    @wraps(func)
    def wrapper(*args, **kwargs):
        result = func(*args, **kwargs)
        return result

    return wrapper
```

## B1 — Pandas Cleaning

```
df["text"] = df["text"].str.strip().str.lower()
df["age"] = df["age"].fillna(df["age"].mean())
df = df.drop_duplicates()
```

## B2 — NumPy Vectorization

```
scores = np.array(...)
result = np.clip(scores + 5, 0, 100)
```

## B3 — Cosine Similarity

```
from sklearn.metrics.pairwise import cosine_similarity
```

```
similarities = cosine_similarity(new_customer, customers)[0]
```

## B4 — Classification Metrics

```
confusion_matrix(...)  
precision_score(...)  
recall_score(...)  
f1_score(...)
```

For the "spam" class:

```
pos_label="spam"
```

## B5 — Token Estimation

```
estimated_tokens = len(text) // 4
```

```
available_budget = max_context - reserved_for_output
```

```
if total_tokens > available_budget:  
    raise ValueError(...)
```